¡Ya casi empezamos!



Centro de e-Learning

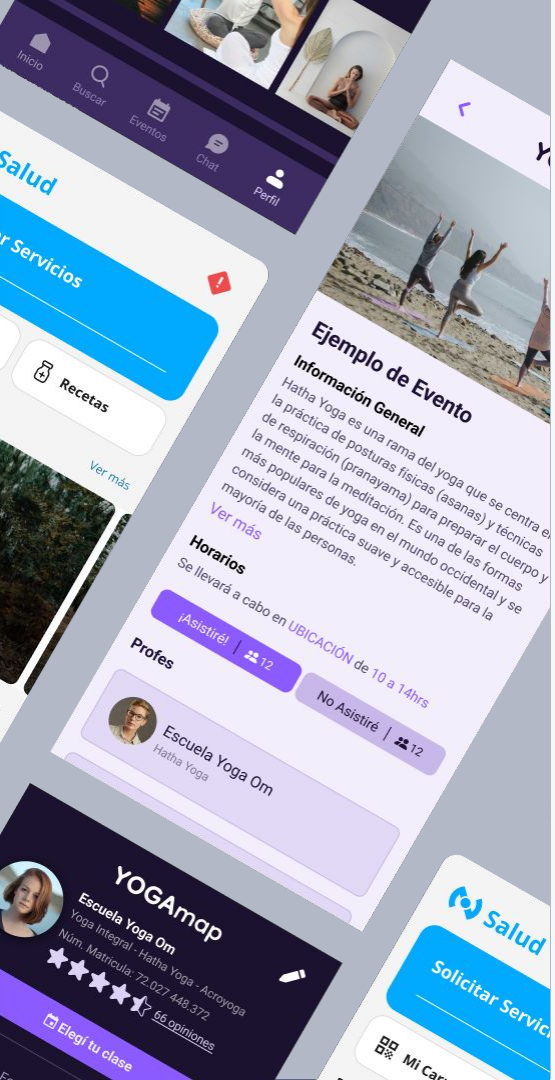
UTN.BA

Clase 1

Fundamentos de React y entornos de trabajo

Centro de e-Learning

UTN.BA

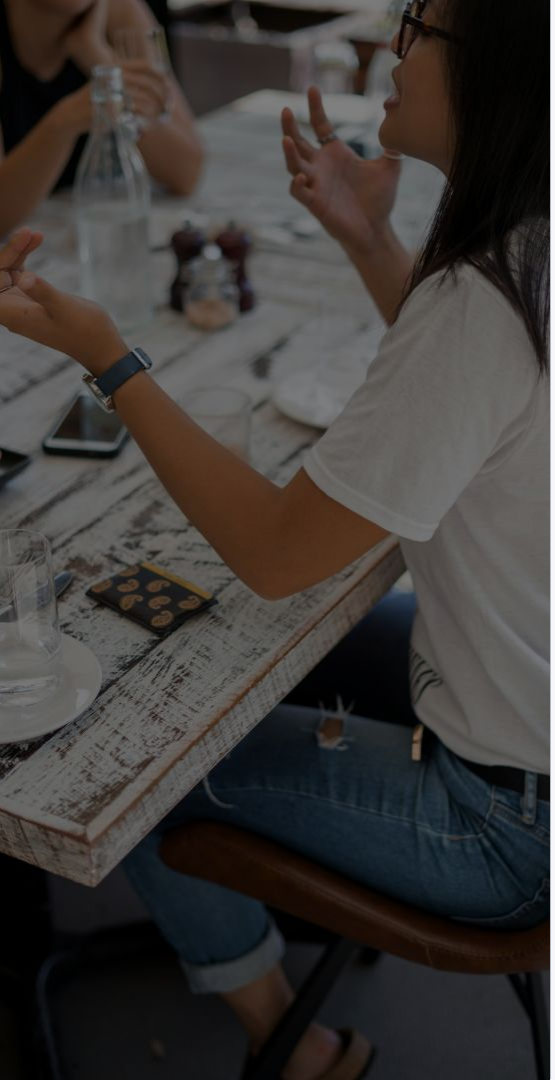


Santiago Rada

¿A qué me dedico?

- Diseño UX/UI
- Frontend Developer
- Backend Developer
- Arquitectura de Información
- Game Design

Centro de e-Learning
UTN.BA

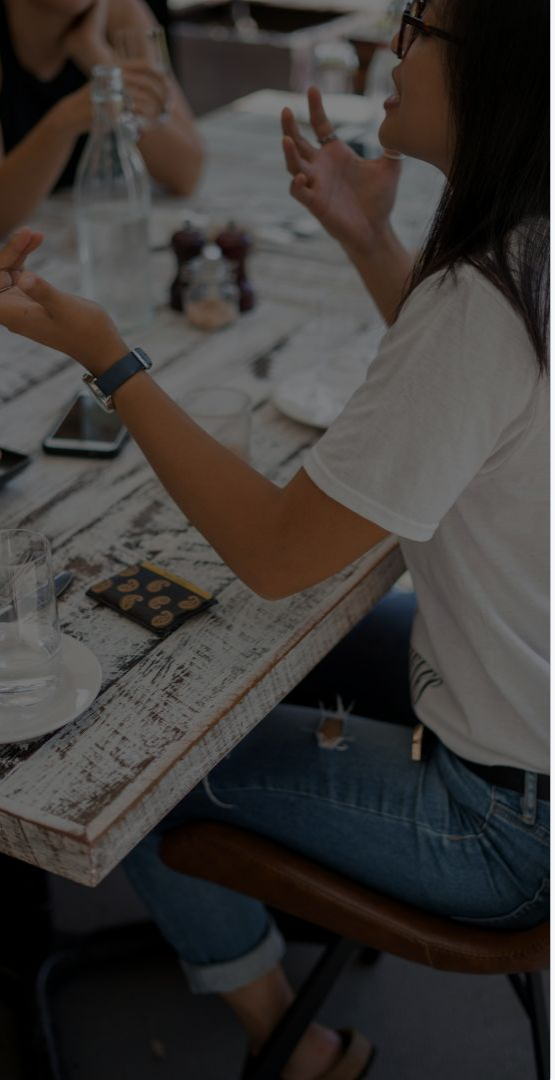


Presentaciones

¿Conocimientos Previos?

Centro de e-Learning

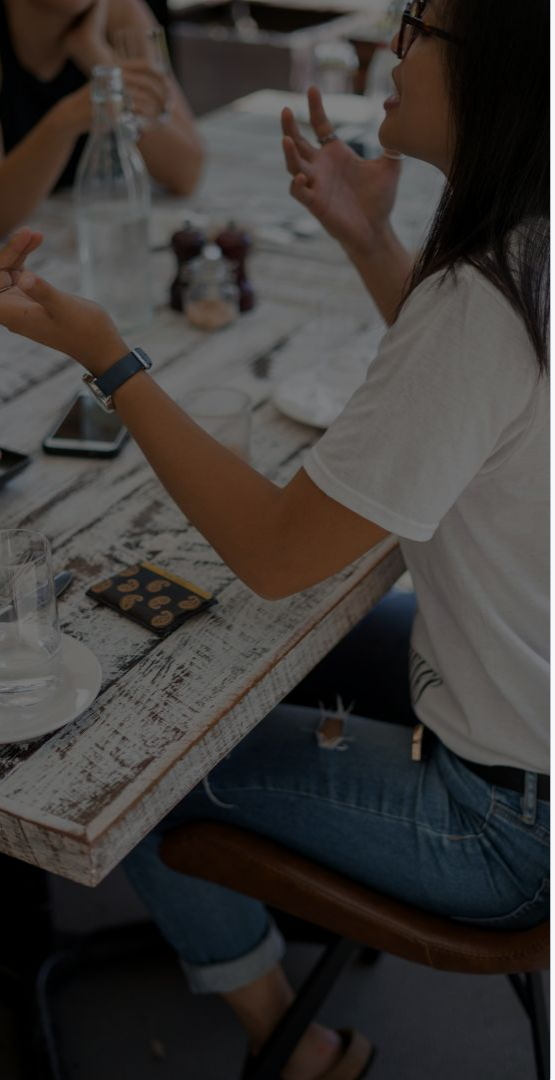
UTN.BA



Presentaciones

¿Por qué elegiste este Curso?

Centro de e-Learning
UTN.BA



Presentaciones

¿Qué expectativa tenés?

¿Qué esperas hacer/aprender?

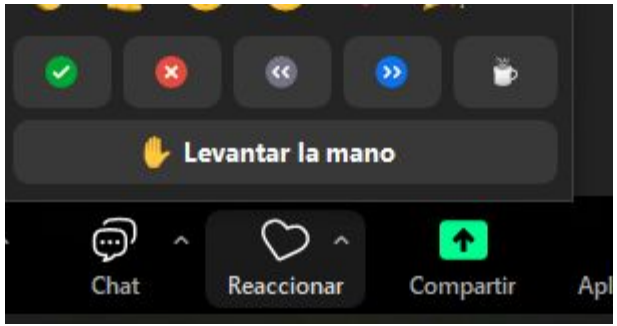
Centro de e-Learning

UTN.BA

Normas de Convivencia

Las clases serán 20% teóricas / 80% prácticas. Siempre que surjan dudas sobre un tema o inciso ya mencionado, pueden levantar la mano desde Zoom en las opciones de reacciones.

Si no cuentan con micrófono también pueden dejar sus dudas en el chat, al cual estaremos muy pendientes.



Presentación de Tutores

Además de las clases en vivo, si tienes dudas sobre un tema, horarios o entregas, puedes consultarlo con los tutores de la comisión, a través del foro de preguntas del campus.



Repasemos el Contenido

Centro de e-Learning
UTN.BA

Entregas y Unidades

1. Fundamentos de React y entorno de trabajo
2. Primeros pasos con React Native y Expo
3. Estado, eventos y ciclo de renderizado
4. Navegación y lineamientos de Material Design
5. Consumo de APIs REST en RN
6. Hooks intermedios y composición lógica
7. Theming y gestión global de estilos
8. Acceso a capacidades nativas del dispositivo
9. Persistencia e integración Backend-as-a-Service

Entregas y Unidades

10. Navegación avanzada y transiciones animadas
11. Arquitectura, navegación y animación
12. Integración de APIs y refinamiento
13. Testing, builds y publicación en tiendas
14. Documentación, pitch y presentación

Información Importante

¿Dónde puedo ver las slides?

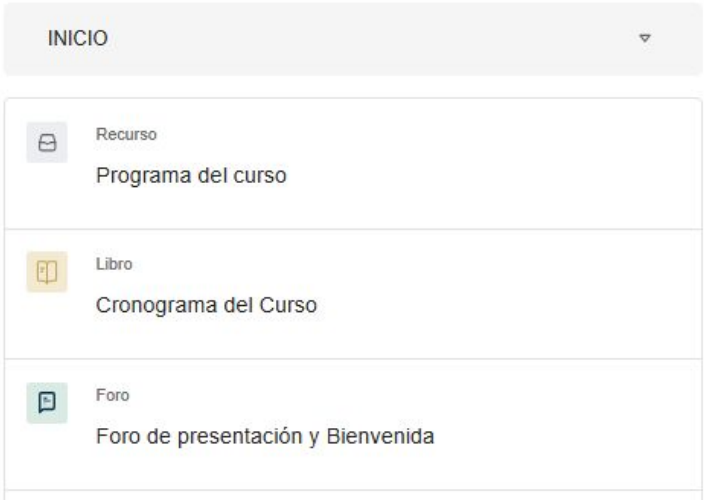
Desde el campus, siempre tendrás acceso al contenido de cada clase:

- ★ Presentaciones (Slides)
- ★ Info de las Entregas
- ★ Espacio para Dudas
- ★ Espacios de Entrega
- ★ Tutoriales cortos

Información de Entregas

¿Dónde puedo ver las Entregas?

También desde el campus, siempre podrás acceder en la pestaña INICIO a la descripción y rubricas del Proyecto Final. El resto de entregas que sucedan en clases intermedias NO son obligatorias pero siempre se recomienda hacerlas.



¿Dudas con el Contenido?

Centro de e-Learning

UTN.BA

Unidad 1

Fundamentos de React y entornos de trabajo

Centro de e-Learning
UTN.BA

Objetivos de la Clase

- **Repasar** puntos clave y fundamentos sobre Javascript previo a comenzar con React Native.
- **Comprender** el contenido básico y principal de React previo a React Native.

Repaso de Contenido
Javascript

Centro de e-Learning
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Variables y scope

Evita **var**. Usa **let** para variables mutables y **const** para valores inmutables; cambian el scope y el hoisting entre ellas.

```
// ❌ Evitar var (function scope)
var nombre = "Juan";

// ✅ Usar let para variables mutables (block scope)
let edad = 25;
edad = 26; // Permite reasignación

// ✅ Usar const para valores inmutables (block scope)
const PI = 3.14159;
const usuario = { nombre: "Ana", edad: 30 };
```

Arrow Functions

Sintaxis corta, sin **this** propio ni uso como constructor; ideales para callbacks y funciones breves.

```
// Función tradicional
function sumar(a, b) {
    return a + b;
}

// Arrow function
const sumar = (a, b) => a + b;

// Arrow function con múltiples líneas
const procesarUsuario = (usuario) => {
    const nombreCompleto = `${usuario.nombre}
${usuario.apellido}`;
    return {
        ...usuario,
        nombreCompleto
    };
```

Template literals

Backticks + ${} para interpolar variables, soportan multi línea y facilitan armar HTML sin concatenar.

```
const nombre = "María";
const edad = 28;

// ❌ Concatenación tradicional
const mensaje1 = "Hola, soy " + nombre + " y tengo " + edad + "
años";

// ✅ Template literals
const mensaje2 = `Hola, soy ${nombre} y tengo ${edad} años`;

// Múltiples líneas
const html = `
    <div>
        <h1>${nombre}</h1>
        <p>Edad: ${edad}</p>
    </div>
`;
```

Destructuring

Extrae datos de objetos y arrays con renombrado, valores por defecto y anidación.

```
const usuario = {
    nombre: "Carlos",
    email: "carlos@email.com",
    edad: 32,
    direccion: {
        ciudad: "Madrid",
        pais: "España"
    }
};

// Destructuring básico
const { nombre, email } = usuario;

// Con renombrado
const { nombre: nombreUsuario, edad: años } = usuario;

// Destructuring anidado
const { direccion: { ciudad } } = usuario;

// Con valores por defecto
const { telefono = "No especificado" } = usuario;
```

Spread y Rest

Copia/combina arrays u objetos y expande argumentos; con Rest agrupas parámetros variables en funciones.

```
// Con arrays
const numeros = [1, 2, 3];
const masNumeros = [...numeros, 4, 5];

// Con objetos
const persona = { nombre: "Ana", edad: 25 };
const personaCompleta = {
    ...persona,
    email: "ana@email.com",
    activo: true
};

// En llamadas a funciones
const max = Math.max(...numeros);
```

Métodos de array

map, filter, find, reduce, some y every permiten transformar, filtrar y validar datos de forma funcional.

Promises y async/await

Promises manejan asincronía con then/catch; async/await simplifica el flujo y el manejo de errores con try/catch.

```javascript
const obtenerUsuario = (id) => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (id > 0) {
                resolve({ id, nombre: `Usuario ${id}` });
            } else {
                reject(new Error("ID inválido"));
            }
        }, 1000);
    });
};

// Usar promises
obtenerUsuario(1)
    .then(usuario => console.log(usuario))
    .catch(error => console.error(error));
```

```javascript
// Función async
const obtenerDatosUsuario = async (id) => {
    try {
        const usuario = await obtenerUsuario(id);
        const permisos = await obtenerPermisos(usuario.id);
        return { ...usuario, permisos };
    } catch (error) {
        console.error("Error:", error.message);
        throw error;
    }
};

// Usar async/await
const cargarDatos = async () => {
    const datos = await obtenerDatosUsuario(1);
    console.log(datos);
};
```

Módulos ES6

Organiza código con export/import (default, nombrados o alias) para reutilizar funciones y constantes.

```js
// utils.js
export const PI = 3.14159;

export const calcularArea = (radio) => PI * radio * radio;

export default class Calculadora {
    static sumar(a, b) {
        return a + b;
    }
}
```

```js
// app.js
import Calculadora, { PI, calcularArea } from './utils.js';

// Import con alias
import { calcularArea as area } from './utils.js';

// Import todo
import * as Utils from './utils.js';
```

¡Es hora de un descansito!

Volvemos en *10'* a seguir *Practicando*



Centro de e-Learning

UTN.BA

Componentes y Props

Centro de e-Learning
UTN.BA

Componentes Funcionales

Funciones que retornan JSX; reciben props y permiten construir interfaces reutilizables.

```jsx
// Componente con props
const Usuario = ({ nombre, edad, activo = true }) => {
    return (
        <div className={`usuario ${activo ? 'activo' :
'inactivo'}`}>
            <h2>{nombre}</h2>
            <p>Edad: {edad}</p>
            <span>{activo ? "En línea" : "Desconectado"}</span>
        </div>
    );
};

// Uso del componente
const App = () => {
    return (
        <div>
            <Usuario nombre="Ana" edad={25} />
            <Usuario nombre="Carlos" edad={30} activo={false} />
        </div>
    );
};
```

JSX: Reglas Básicas

Un único elemento padre, atributos camelCase, etiquetas cerradas y JavaScript entre {} para lógica y estilos.

Props y Childrens

Las props pasan datos y callbacks; children inyecta contenido interno flexible. Se pueden definir valores por defecto.

```
        <button
            type="button"
            className={`btn btn-${tipo}`}
            onClick={onClick}
            disabled={deshabilitado}
        >
            {texto}
        </button>
    );
};

// Props children
const Modal = ({ children, visible, onClose }) => {
    if (!visible) return null;

    return (
        <div className="modal-overlay" onClick={onClose}>
            <div className="modal-content" onClick={e =>
e.stopPropagation()}>
                <button className="close"
```

React con JSX:
Hooks

Centro de
e-Learning
UTN.BA

useState

Hook para estado local. Actualiza valores (primitivos u objetos) sin mutar el estado previo.

```
import { useState } from 'react';

const Contador = () => {
    const [count, setCount] = useState(0);

    const incrementar = () => setCount(count + 1);
    const decrementar = () => setCount(count - 1);
    const reset = () => setCount(0);

    return (
        <div className="contador">
            <h2>Contador: {count}</h2>
            <div className="botones">
                <button onClick={decrementar}>-</button>
```

useEffect

Ejecuta efectos tras el render: fetch, timers o limpieza al desmontar. Controla cuándo correr con el array de dependencias.

```jsx
import { useState, useEffect } from 'react';

// Efecto al montar y desmontar
const ComponenteConEfecto = () => {
    const [datos, setDatos] = useState([]);
    const [cargando, setCargando] = useState(true);

    useEffect(() => {
        // Efecto (ejecuta al montar)
        console.log("Componente montado");

        const cargarDatos = async () => {
            try {
                const response = await fetch('/api/datos');
                const data = await response.json();
                setDatos(data);
            } catch (error) {
                console.error("Error cargando datos:", error);
```

React con JSX VS
React Native

Centro de
e-Learning
UTN.BA

Diferencias React VS React Native

React web usa y CSS; React Native usa <View>, <Text> y StyleSheet. Cambian eventos y navegación, pero el layout sigue basado en Flexbox.

Conceptos que se mantienen

Componentes funcionales, hooks, props/state, JSX y ciclo de vida se usan igual en RN.

Novedades en React Native

Componentes nativos, StyleSheet, navegación entre pantallas, APIs del dispositivo y gestos táctiles requieren nuevas prácticas.

Documentación oficial

Consulta MDN para JS y la documentación oficial de React y React Native para profundizar conceptos y hooks.

Herramientas
Extra

Herramientas Recomendadas

- VS Code
- NodeJS
- Git
- Chrome DevTools como base del entorno de desarrollo.

NodeJS

Es un entorno de ejecución de JavaScript fuera del navegador. Permite ejecutar código JS en el servidor gracias a su motor V8 de Google Chrome.

NodeJS

Con NodeJS los desarrolladores pueden crear aplicaciones backend, scripts, herramientas de línea de comandos, y ejecutar código que normalmente solo correría en el navegador.

NPM

Es el gestor de paquetes que viene incluido con Node.js. Permite instalar, actualizar y gestionar librerías y dependencias de un proyecto.

NPM

Este gestor es necesario para instalar React, ReactDOM, otras dependencias y para ejecutar scripts como ***npm start*** o ***npm build***.

Extensiones VS Code

- Snippets ES7+
- Prettier
- ESLint
- Bracket Pair Colorizer
- Auto Rename Tag

Aceleran el flujo y mejoran la calidad del código.

¿Preguntas?
Debatamos un poco los temas de hoy.



Centro de e-Learning
UTN.BA

¡Nos vemos la próxima clase!

¡Primeros pasos con React Native y Expo!

Centro de e-Learning
UTN.BA

Bibliografía

- **Mozilla Developer Network.** *JavaScript: MDN Web Docs*. [Internet]. [sin edición]. Estados Unidos: Mozilla Foundation; [consultado en 2025]. Disponible en: https://developer.mozilla.org/es/docs/Web/JavaScript
- **Meta Platforms, Inc**. *React – A JavaScript library for building user interfaces*. [Internet]. [sin edición]. Estados Unidos: Meta; [consultado en 2025]. Disponible en: https://react.dev/
- **Meta Platforms, Inc.** *React Native – Learn once, write anywhere*. [Internet]. [sin edición]. Estados Unidos: Meta; [consultado en 2025]. Disponible en: https://reactnative.dev/
- **Mozilla Developer Network**. *Array - JavaScript | MDN*. [Internet]. [sin edición]. Estados Unidos: Mozilla Foundation; [consultado en 2025]. Disponible en: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array
- **Mozilla Developer Network.** *Promise - JavaScript | MDN*. [Internet]. [sin edición]. Estados Unidos: Mozilla Foundation; [consultado en 2025]. Disponible en: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise
- **Meta Platforms, Inc**. *React Docs: useState Hook*. [Internet]. [sin edición]. Estados Unidos: Meta; [consultado en 2025]. Disponible en: https://react.dev/reference/react/useState
- **Meta Platforms, Inc**. *React Docs: useEffect Hook*. [Internet]. [sin edición]. Estados Unidos: Meta; [consultado en 2025]. Disponible en: https://react.dev/reference/react/useEffect